



ICT306 Advanced Cybersecurity

WEEK 4(A) Cryptography

Exploring the foundations of cryptographic systems — from classical ciphers to modern encryption standards.



Resources

Prescribed Reading

Stallings, W. (2023). *Cryptography and Network Security: Principles and Practice* (8th ed., Global ed.). Pearson.

Recommended Reading

Whitman, M. E., & Mattord, H. J. (2021). *Principles of Information Security*. Cengage Learning.

Module 9 is the relevant chapter for this lecture's content.

Lecture Overview

This lecture covers the core principles and techniques of cryptography, progressing from foundational concepts to advanced key exchange mechanisms.

01

Basic Concepts

Definitions, objectives, and the cryptographic process.

03

Asymmetric Key Encryption

RSA algorithm and Diffie-Hellman key exchange.

02

Symmetric Key Encryption

Classical ciphers, random numbers, stream ciphers, block ciphers, and modes of operation.

04

DNSSEC

Applying cryptography to secure the Domain Name System.

What Is Cryptography?

The word **cryptography** derives from the Greek *kryptos* ("concealed, hidden, veiled, secret, or mysterious") and *graphia* ("writing") — literally, "**the art of secret writing.**"

Definition

The practice of concealing information by converting **plaintext** (readable format) into **ciphertext** (unreadable format) using a key or encryption scheme.

What It Protects

- Email messages and chat sessions
- Web transactions and e-commerce
- Personal and corporate data
- Government and financial communications



Objectives of Cryptography

Cryptography serves four fundamental security objectives that together form the backbone of secure communication systems.



Confidentiality

Assurance that information is accessible only to those authorized to access it.



Integrity

Trustworthiness of data by preventing improper and unauthorized changes.



Authentication

Assurance that a communication, document, or data is genuine and from a trusted source.



Nonrepudiation

Guarantee that neither sender nor recipient can deny involvement in a transmitted message.

The Cryptographic Process

Cryptography transforms readable plaintext into unintelligible ciphertext at the sender's end and reverses that transformation at the receiver's end. The security of this process depends on the algorithm and the secrecy of the key.

Symmetric Encryption

Both parties share the same key for encryption and decryption.

Asymmetric Encryption

A public/private key pair is used — one to encrypt, one to decrypt.

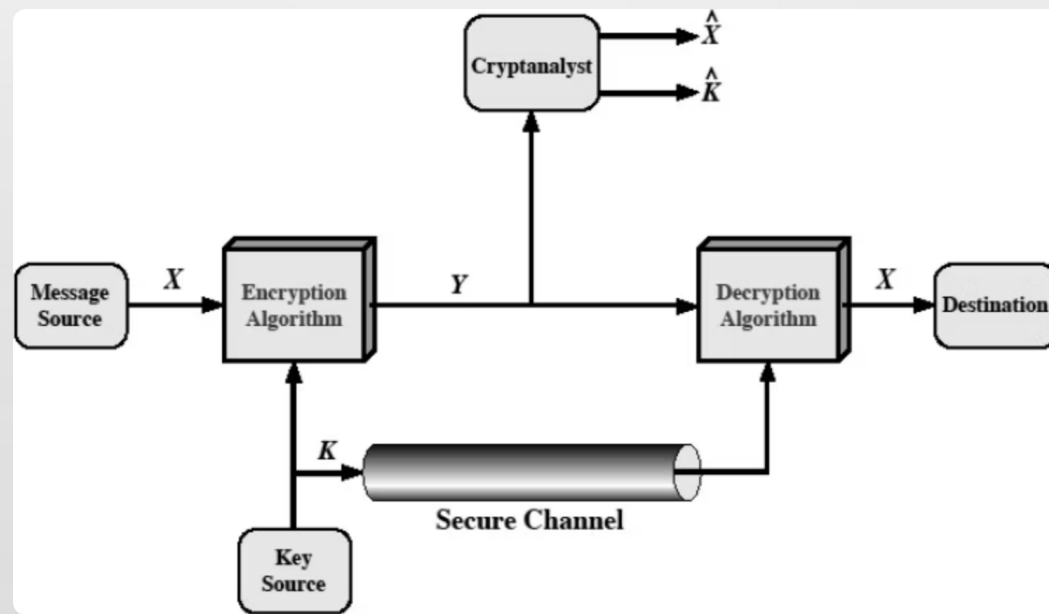


Symmetric Key Encryption

In symmetric key encryption, **both the sender and receiver use the same secret key** to encrypt and decrypt data. This approach is fast and efficient but requires a secure mechanism to exchange the shared key before communication begins.

- 📄 **Key Challenge:** Symmetric encryption requires a secure out-of-band mechanism for key distribution. If the key is intercepted during exchange, the entire communication is compromised.





Model of a Symmetric Cryptosystem

This model illustrates the complete flow of a symmetric encryption system. A message source generates plaintext X , which is encrypted using algorithm E and key K to produce ciphertext Y . The decryption algorithm reverses this using the same key. A cryptanalyst may attempt to recover X or K by analyzing the intercepted ciphertext over the channel.

Attacks on Conventional Encryption

Two primary categories of attacks target conventional encryption schemes. Understanding both is essential for evaluating a cipher's real-world security.

Cryptanalysis

- **Exploits weaknesses** in the structure of the algorithm itself.
- The attacker **uses knowledge of the algorithm** along with **plaintext samples or plaintext–ciphertext pairs** to deduce the key or plaintext.

Brute-Force Attack

- Systematically tries every possible key on a piece of ciphertext until an intelligible plaintext is recovered.
- On average, **half of all possible keys** must be tried before success.
- Effectiveness is proportional to key length.

Types of Cryptanalytic Attacks

Cryptanalytic attacks are classified by the amount of information available to the attacker. More available information generally makes the attack more powerful.

Type of Attack	Known to Cryptanalyst
Ciphertext Only	• Encryption algorithm • Ciphertext
Known Plaintext	• Encryption algorithm • Ciphertext • One or more plaintext–ciphertext pairs formed with the secret key
Chosen Plaintext	• Encryption algorithm • Ciphertext • Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key
Chosen Ciphertext	• Encryption algorithm • Ciphertext • Ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key
Chosen Text	• Encryption algorithm • Ciphertext • Plaintext message chosen by cryptanalyst, together with its corresponding ciphertext generated with the secret key • Ciphertext chosen by cryptanalyst, together with its corresponding decrypted plaintext generated with the secret key

Unconditional vs. Computational Security

Unconditional Security

An encryption scheme is unconditionally secure if the ciphertext provides **insufficient information** to uniquely determine the corresponding plaintext — regardless of the computing power or time available.

The cipher **cannot** be broken under any circumstances.

Computational Security

An algorithm is computationally secure if it meets **one or both** of the following criteria:

1. The cost of breaking the cipher exceeds the value of the encrypted information.
2. The time required to break the cipher exceeds the useful lifetime of the information.

Given limited computing resources, the cipher cannot be broken in practice.

Table 2.2 Average Time Required for Exhaustive Key Search

Key Size (bits)	Number of Alternative Keys	Time Required at 1 Decryption/ μ s	Time Required at 10^6 Decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31}\mu\text{s} = 35.8$ minutes	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55}\mu\text{s} = 1142$ years	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127}\mu\text{s} = 5.4 \times 10^{24}$ years	5.4×10^{18} years
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167}\mu\text{s} = 5.9 \times 10^{36}$ years	5.9×10^{30} years
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26}\mu\text{s} = 6.4 \times 10^{12}$ years	6.4×10^6 years

Brute-Force Attack: Key Size Matters

- A brute-force attack is always theoretically possible — the attacker simply tries every key.
- The critical assumption is that the attacker can recognize valid plaintext when found.
- The table below demonstrates how key size dramatically affects the feasibility of such an attack.

❏ A 128-bit key produces $2^{128} \approx 3.4 \times 10^{38}$ possible keys — infeasible to brute-force with any foreseeable computing technology.

Encryption Algorithms & Ciphers

Encryption converts readable plaintext into unreadable ciphertext using complex algorithms that transform data into blocks or streams of random alphanumeric characters.

Cipher

An algorithm — a series of well-defined steps — for performing encryption and decryption.

Encipherment

The process of converting plaintext into ciphertext using the cipher and a key.

Decipherment

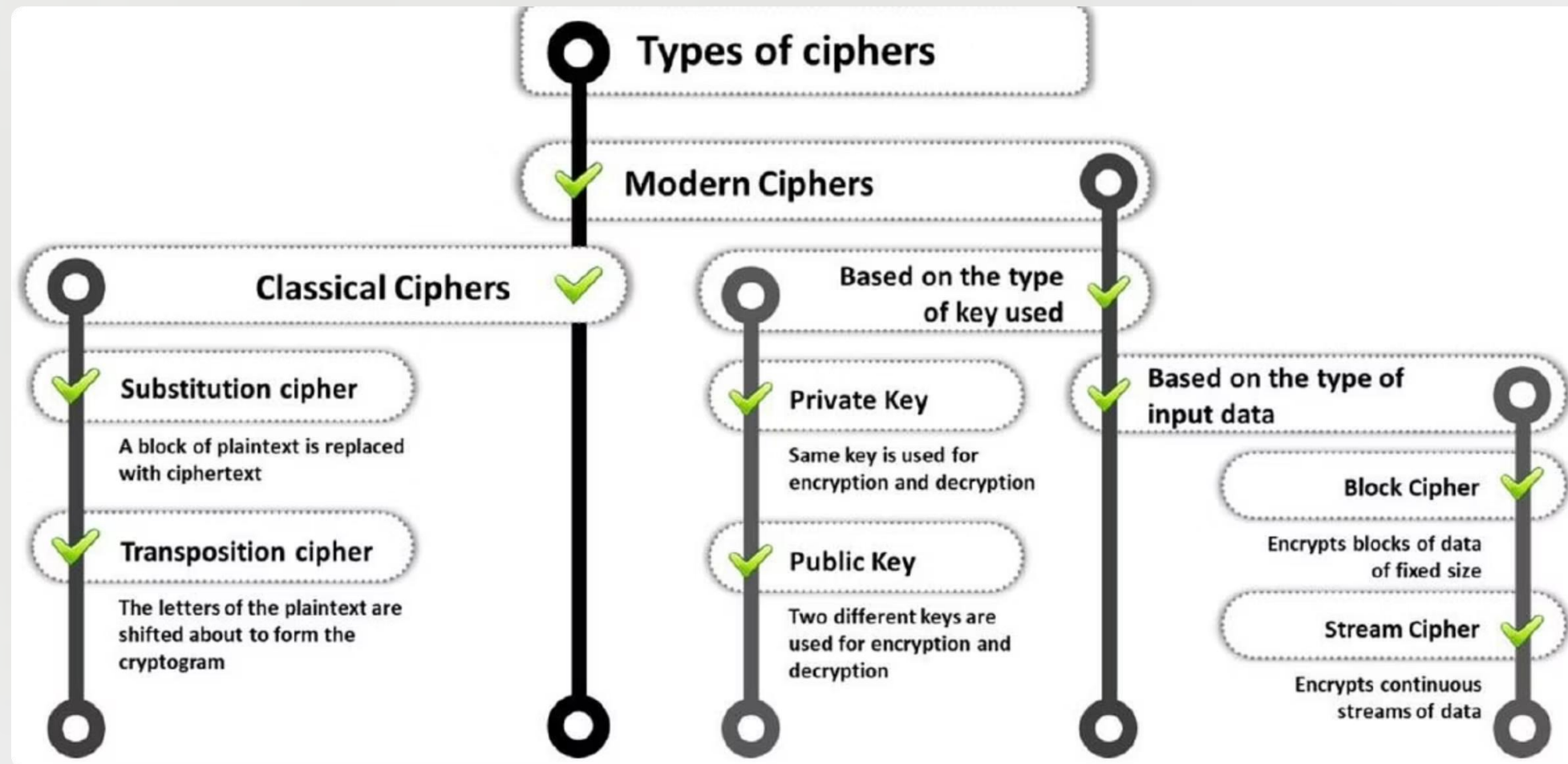
The reverse process: converting ciphertext back into readable plaintext.

Open vs. Closed Source

Ciphers may be open-source (publicly available) or closed-source (proprietary, e.g., military). They may also be free or commercially licensed.

Types of Cipher

Ciphers are broadly categorized into classical and modern types, further subdivided based on key structure and data input type.



Substitution Cipher

- Classical ciphers operate on letters of the alphabet (A–Z).
- Because they are **easily deciphered using frequency analysis**, they are considered **cryptographically unreliable** by modern standards.

How It Works

- Each unit of plaintext (single letter, pair, or combination) is replaced with a corresponding ciphertext unit according to a fixed system.
- The recipient reverses the substitution to decrypt.

Examples: Caesar cipher, Vigenère cipher, Playfair cipher, Hill cipher.

Example

The plaintext **"HELLO WORLD"** might encrypt to **"PSTER HGFST"**, where each letter maps to a different letter (H→P, E→S, L→T, etc.).

CLASSICAL CIPHERS

Transposition Cipher

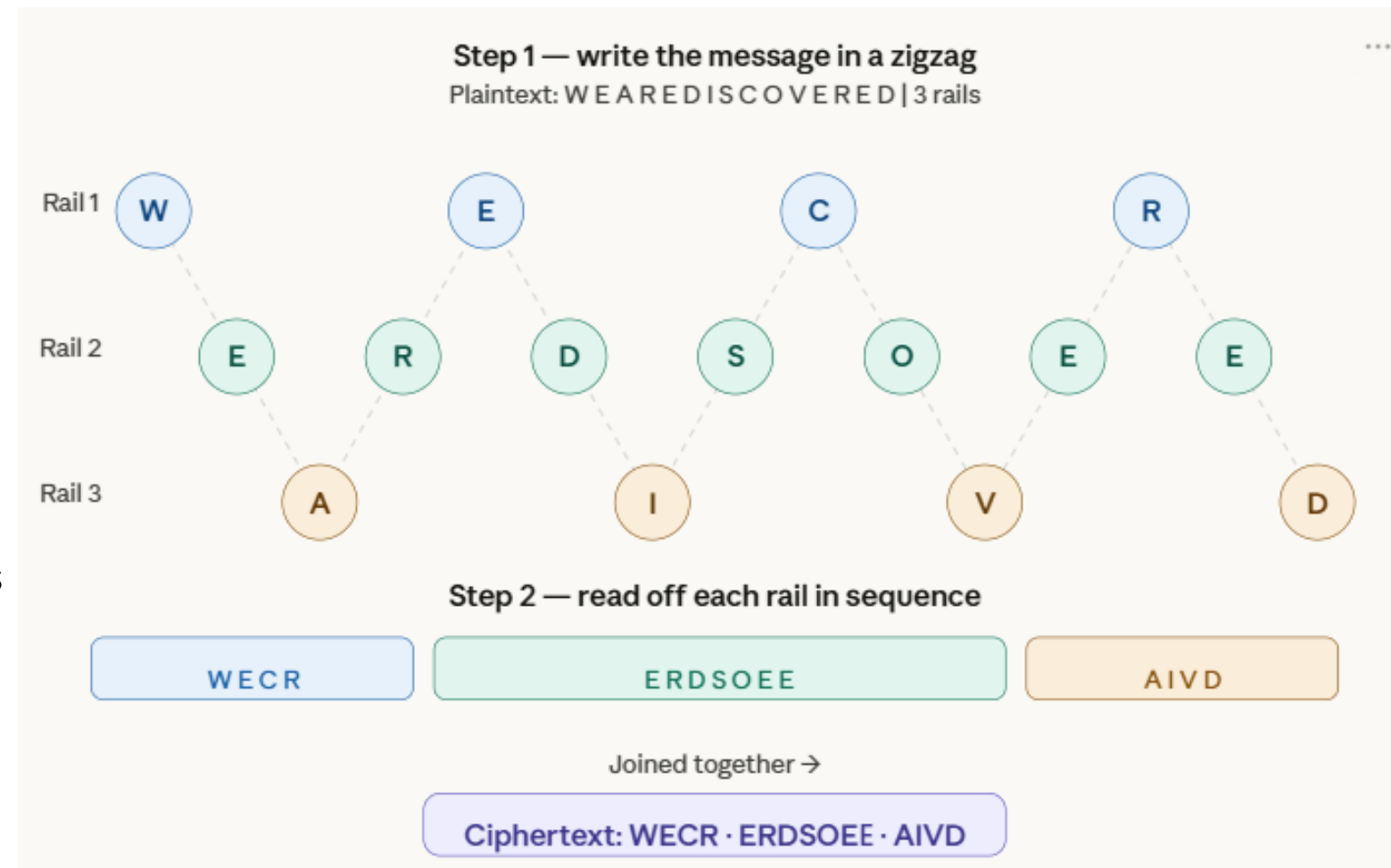
In a **transposition cipher**, the letters of the plaintext are not replaced — they are **rearranged** according to a regular system to produce ciphertext. The message content remains the same, but its order is scrambled.

Example

The plaintext "**CRYPTOGRAPHY**" when encrypted using a transposition scheme becomes "**AOYCRGPTYRHP**" — same letters, different positions.

Notable Example

The **Rail Fence Cipher** writes plaintext diagonally across a set of "rails" (rows) and then reads off each rail in sequence to form ciphertext.



CLASSICAL CIPHERS

Caesar Cipher

- The Caesar cipher is among the earliest known and most historically significant substitution ciphers, first attested in Julius Caesar's military communications.
- It works by **replacing each letter with the letter three positions ahead** in the alphabet.

📄 **Example:** "meet me after the toga party" encrypts to "PHHW PH DIWHU WKH WRJD SDUWB" — each letter shifted by 3.

plain: meet me after the toga party
cipher: PHHW PH DIWHU WKH WRJD SDUWB

Caesar Cipher: Mathematical Formulation

- The Caesar cipher can be expressed precisely using modular arithmetic, where each letter is assigned a numeric value (A=0, B=1, ..., Z=25).
- The shift value k is the key.

Encryption

$$c = E(k, p) = (p + k) \bmod 26$$

Decryption

$$p = D(k, c) = (c - k) \bmod 26$$

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Brute-Force Attack on Caesar Cipher

The Caesar cipher is particularly vulnerable to brute-force attack due to its extremely small key space. An attacker can exhaustively recover the plaintext with minimal effort.

Known Algorithm

The encryption and decryption algorithm are publicly known, eliminating algorithmic obscurity as a defense.

Only 25 Keys

There are only 25 possible shift values ($k = 1$ to 25). An attacker can try all of them in seconds.

Recognizable Plaintext

The language of the plaintext is known and easily recognizable, allowing the attacker to identify the correct decryption immediately.



Monoalphabetic Cipher

- A monoalphabetic cipher maps each letter of the plain alphabet to a **unique, fixed cipher alphabet** — but unlike the Caesar cipher, the mapping need not follow a simple shift.
- The letters can be shuffled (jumbled) arbitrarily.

Key Space

There are $26! = 4 \times 10^{26}$ possible mappings — an astronomically large key space that might seem secure at first glance.

The Weakness

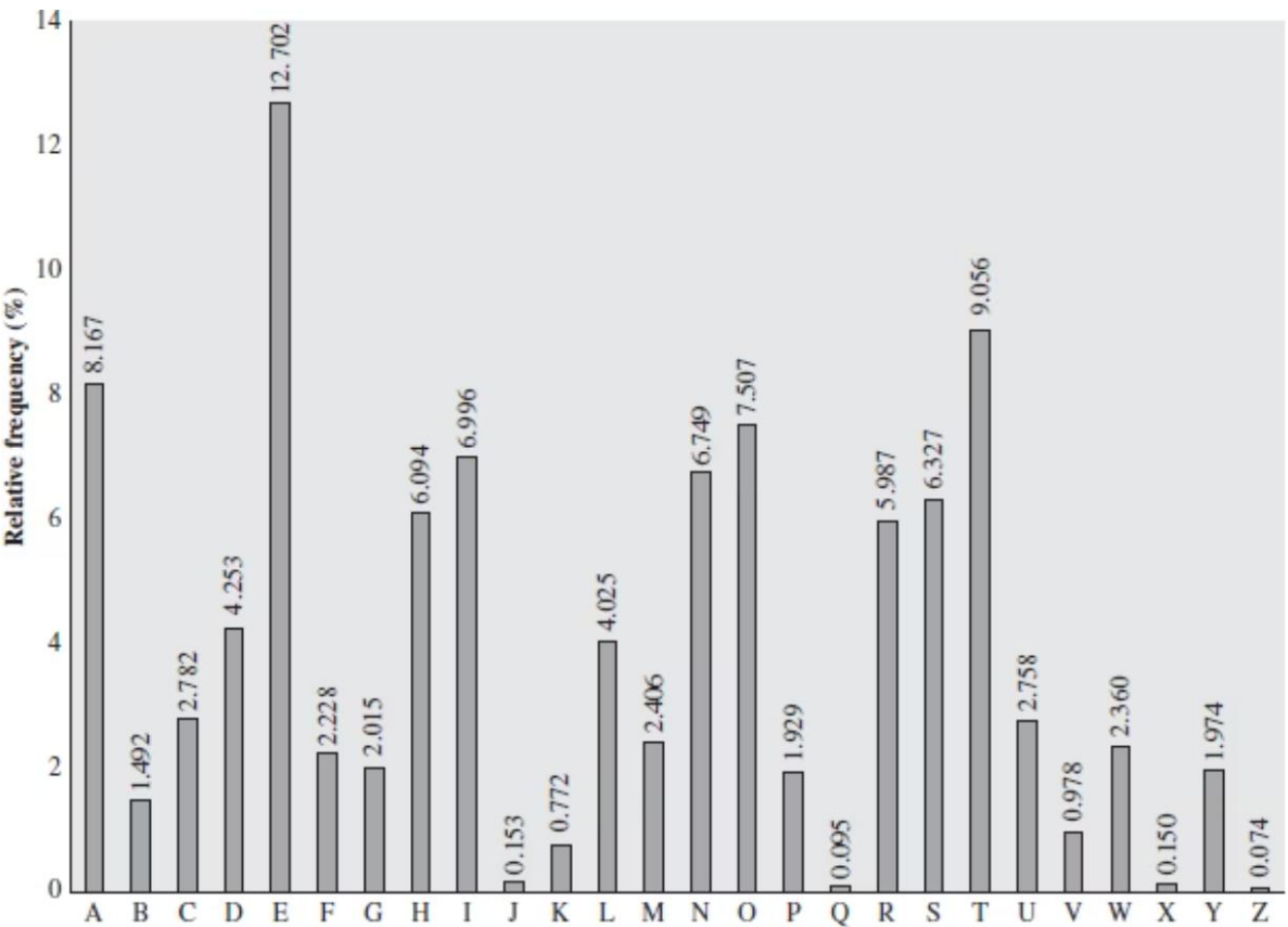
Despite the vast key space, monoalphabetic ciphers are **vulnerable to frequency analysis**.

Language characteristics — letter frequencies, common digrams and trigrams — expose the cipher's structure.

```
plain:  a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```


Language Redundancy & Cryptanalysis

- The **critical vulnerability** of monoalphabetic ciphers lies in **their failure to disguise** the statistical structure of the language.
- **Because each plaintext letter always maps to the same ciphertext letter, relative letter frequencies are preserved** in the ciphertext.



Frequency Analysis Attack

In English, the letter 'E' appears most frequently (~12.7%), followed by 'T', 'A', 'O', and 'I'.

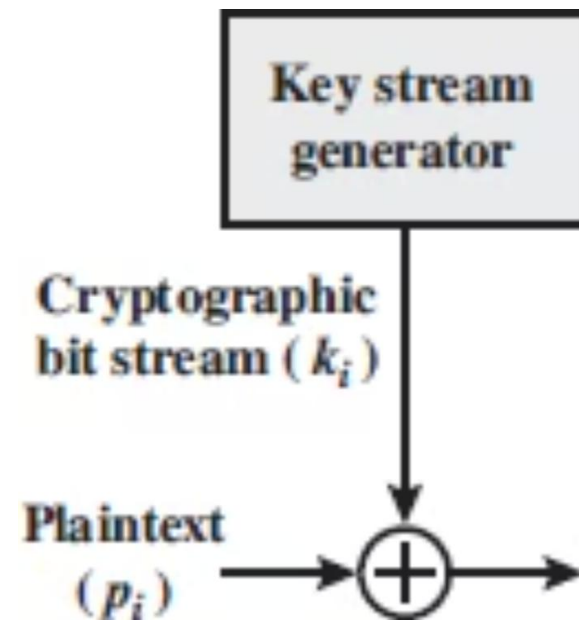
An attacker analyzes ciphertext letter frequencies and **maps the most common ciphertext letters to the most common English letters.**

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMZSHZOWSFPAPPDTSVPQUZWMXUZHUSX
EPYEPDPZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

P 13.33	H 5.83	F 3.33	B 1.67	C 0.00
Z 11.67	D 5.00	W 3.33	G 1.67	K 0.00
S 8.33	E 5.00	Q 2.50	Y 1.67	L 0.00
U 8.33	V 4.17	T 2.50	I 0.83	N 0.00
O 7.50	X 4.17	A 1.67	J 0.83	R 0.00
M 6.67				

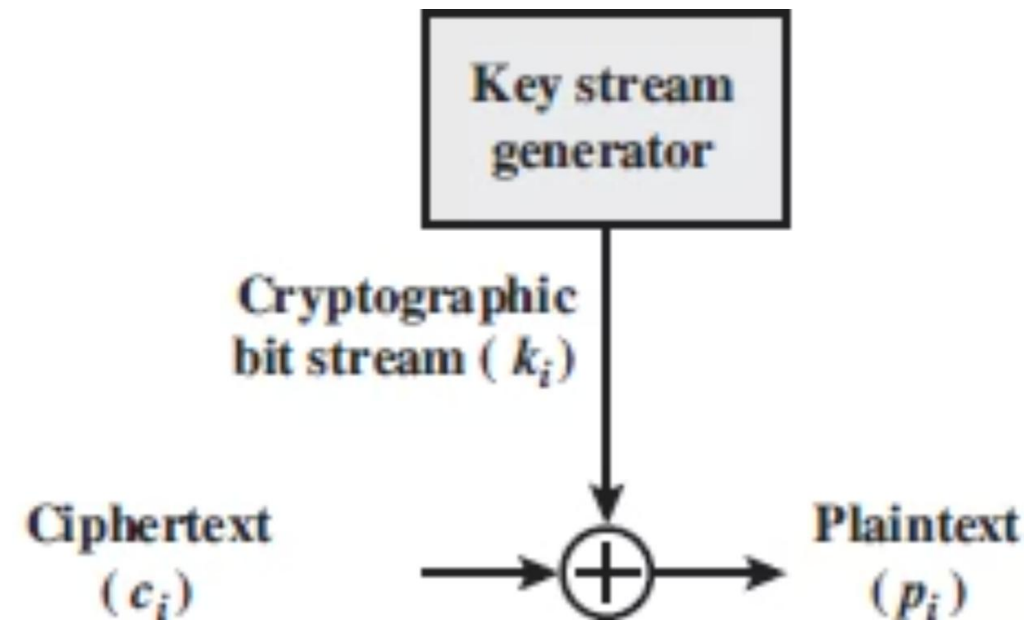
Vernam Cipher

- Invented by AT&T engineer **Gilbert Vernam** in **1918**, the Vernam cipher was a landmark development in cryptography.
- It applies a keystream using the **XOR (exclusive OR)** operation on each bit of plaintext — forming the conceptual basis for the one-time pad.



Encryption

$$c_i = p_i \oplus k_i$$



Decryption

$$p_i = c_i \oplus k_i$$



One-Time Pad

The **one-time pad (OTP)** is theoretically the only provably unbreakable encryption scheme in existence, provided it is used correctly.

Truly Random Key

The key must be completely random and at least as long as the plaintext message being encrypted.

Single Use Only

The key is used exactly once and then discarded. Reuse destroys the security guarantee.

Perfect Secrecy

For any plaintext and any ciphertext, there exists a key mapping one to the other — the ciphertext bears no statistical relationship to the plaintext.

Practical Limitation

The generation and secure distribution of truly random keys of sufficient length makes the OTP impractical for most real-world applications.

Random Numbers in Cryptography

Randomness is a foundational requirement in cryptographic systems.

Many cryptographic mechanisms **depend entirely on the quality of the random numbers used.**

Criteria for Randomness

- **Uniform Distribution:** Each value must be equally likely to occur.
- **Independence:** Each generated number must be statistically independent of others.
- **Unpredictability:** Future values must not be derivable from past values.

Cryptographic Uses

- Nonces in authentication protocols (to prevent replay attacks)
- Session key generation
- Public key pair generation
- Keystreams for one-time pads

Care must be taken during the generation of pseudorandom numbers, as **weak PRNGs are a common source of cryptographic vulnerabilities.**

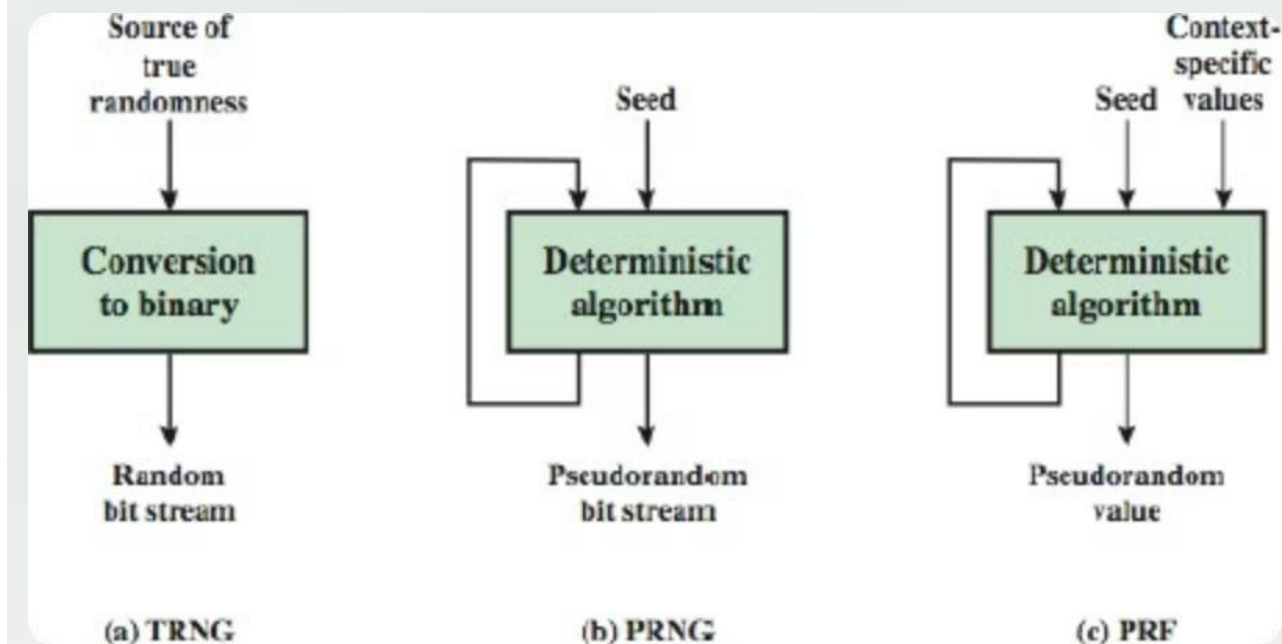
Pseudorandom Number Generators (PRNGs)

True randomness is difficult to achieve computationally.

In practice, cryptographic systems often rely on **deterministic algorithms** that produce sequences of numbers that *appear random* — known as **pseudorandom numbers**.

- ❏ **PRNGs are not truly random** — they can pass many statistical tests of randomness but are ultimately determined by their initial seed value.

Cryptographically Secure PRNGs (CSPRNGs) must also be computationally infeasible to predict.



PRNG: Entropy Sources & Seeds

A **high-quality PRNG** requires a reliable source of entropy to initialize its state.

The quality of the seed directly determines the quality and unpredictability of the generated numbers.

Entropy Source (TRNG)

A **True Random Number Generator (TRNG)** draws from physical environmental events:
keystroke timing, disk electrical activity, mouse movements, and instantaneous system clock values — all fundamentally unpredictable.

Seed

The seed is typically generated by a TRNG and used **to initialize** the PRNG's deterministic algorithm.
It must remain secret.

Critical Vulnerability

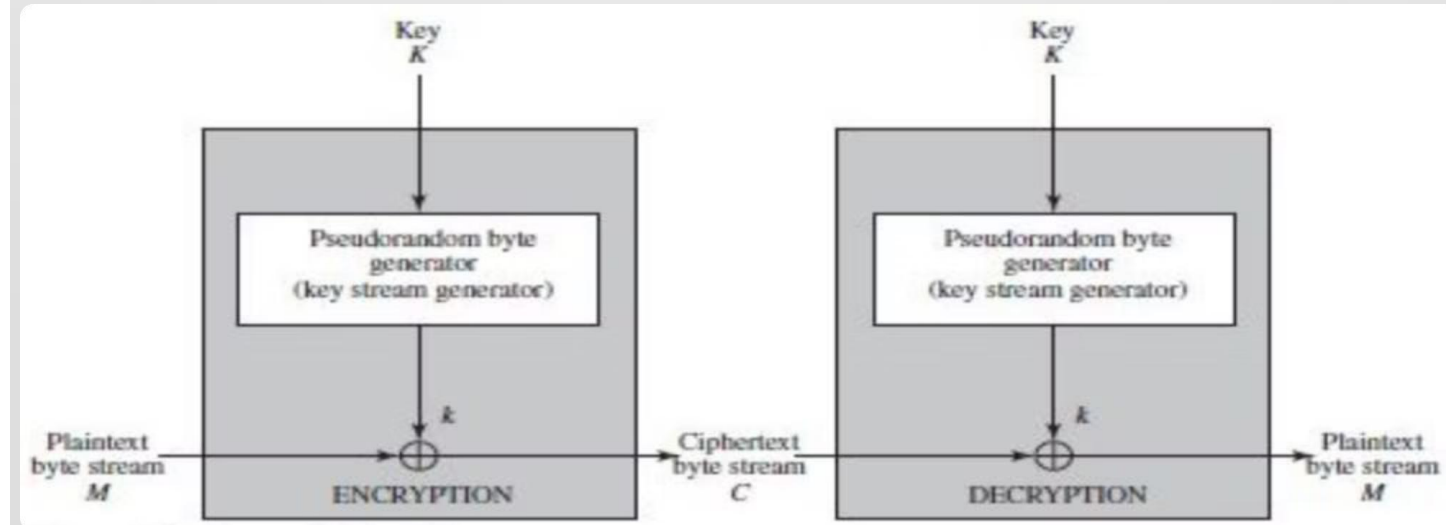
If an adversary knows both the PRNG algorithm **and** the seed, they can reproduce the entire bit stream — **completely defeating the purpose of the PRNG.**

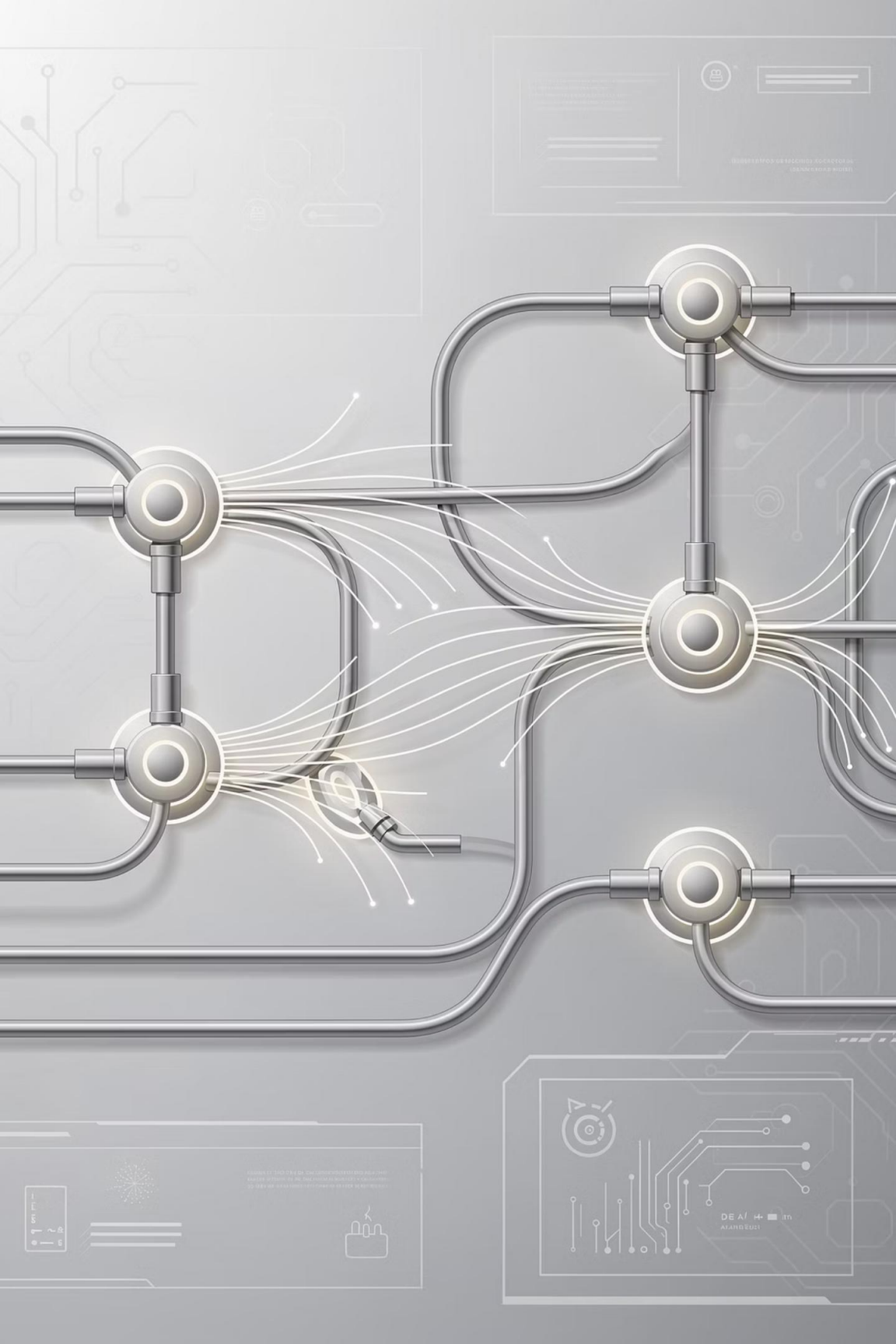
STREAM CIPHERS

Stream Cipher Fundamentals

- A stream cipher encrypts data **one bit (or byte) at a time**, combining a pseudorandom keystream with the plaintext using the XOR operation.
- This approach is particularly **well-suited for real-time data transmission**.

📄 **Core Operation:** $C_i = M_i \text{ XOR } \text{StreamKey}_i$ — each plaintext bit is XOR'd with the corresponding keystream bit to produce the ciphertext bit.





STREAM CIPHERS

Stream Cipher: Critical Rules

Never Reuse the Keystream

- Reusing the same stream key for two different messages is **a fatal mistake**.
- An attacker who **XORs the two ciphertexts together eliminates the keystream**, revealing a combination of the two plaintexts — *often recoverable through linguistic analysis*.

Appropriate Use Cases

Stream ciphers are well-suited for **data communication channels** and **browser/web links** where **continuous, low-latency encryption** is required without fixed-length block boundaries.

RC4 Stream Cipher

- **RC4** (Rivest Cipher 4) is a proprietary stream cipher designed by Ron Rivest and owned by RSA Security.
- Despite its simplicity, [it was one of the most widely deployed stream ciphers in history](#).



Variable Key Size

RC4 supports variable-length keys, typically ranging [from 40 to 256 bits](#), making it flexible across different security requirements.



Widespread Adoption

Widely used in **SSL/TLS** (web security), **WEP/WPA** (wireless security), **SSH**, and **Kerberos**.
[Now considered deprecated due to known vulnerabilities.](#)



Byte-Oriented Design

RC4 operates on bytes rather than bits, contributing to its simplicity and efficiency in software implementations.

RC4 Key Schedule Algorithm

The RC4 Key Schedule Algorithm (KSA) initializes the cipher's internal state array **S** before encryption begins. **This state** is then used to generate the keystream.

- The array **S** starts as an identity permutation (**$S[0]=0, S[1]=1, \dots, S[255]=255$**).
- **The key** is then used to thoroughly shuffle **S**, producing a key-dependent permutation that forms the internal state of the cipher.

```
for i = 0 to 255 do
    S[i] = i
    T[i] = K[i mod keylen]
j = 0
for i = 0 to 255 do
    j = (j + S[i] + T[i]) (mod 256)
    swap (S[i], S[j])
```

STREAM CIPHERS

RC4 Encryption Process

After key scheduling, the Pseudo-Random Generation Algorithm (PRGA) produces the keystream. Each byte of the keystream is XOR'd with the corresponding plaintext byte to produce ciphertext.

The encryption continues by shuffling array values, using the sum of a shuffled pair to select a stream key value **S[t]** from the permutation.

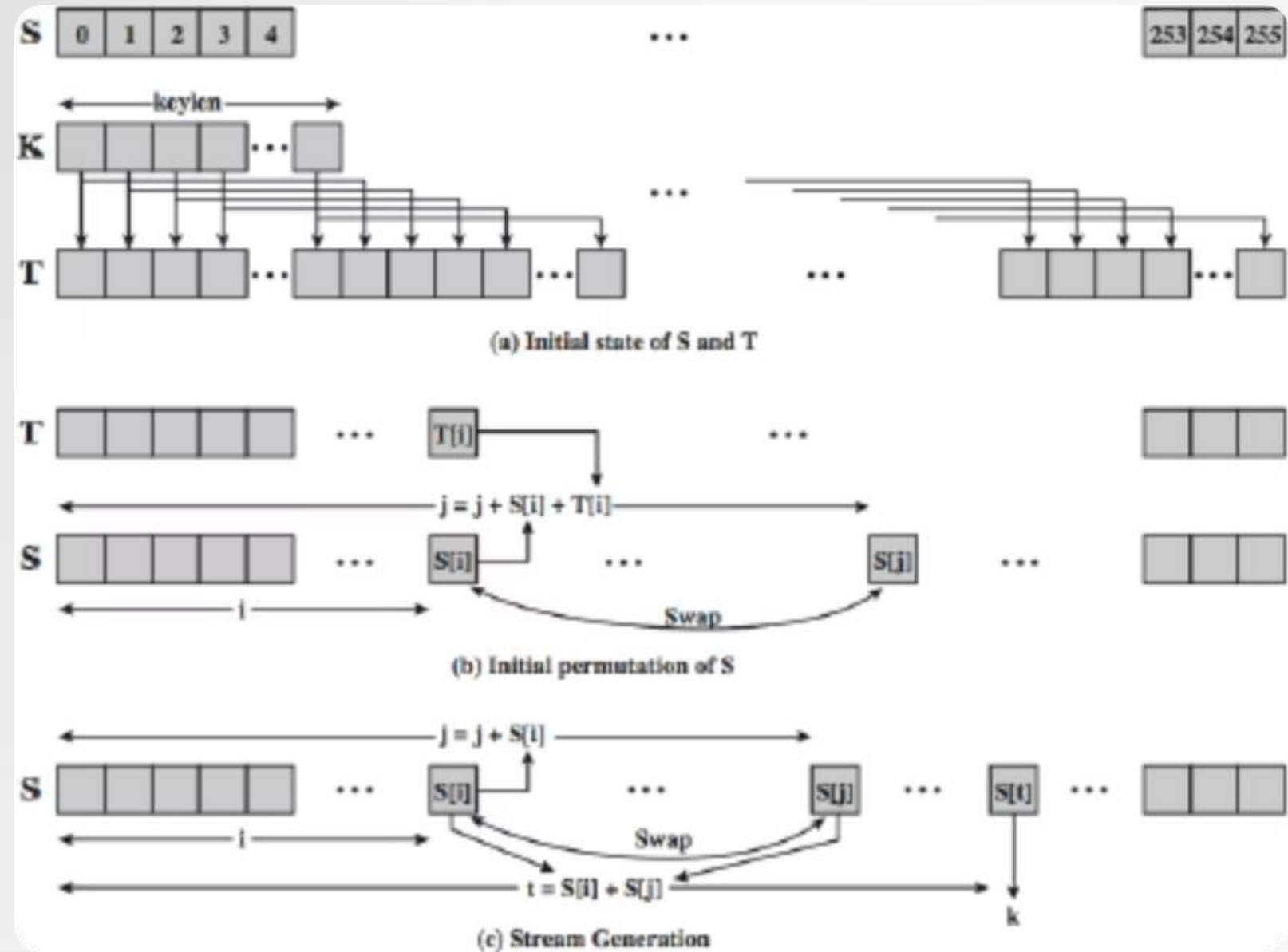
This value is XOR'd with the next byte of the message to encrypt or decrypt. RC4 is used optionally in **SSH** and **Kerberos**.

```
/* Stream Generation */  
i, j = 0;  
while (true)  
    i = (i + 1) mod 256;  
    j = (j + S[i]) mod 256;  
    Swap (S[i], S[j]);  
    t = (S[i] + S[j]) mod 256;  
    k = S[t];
```

STREAM CIPHERS

RC4 Overview Diagram

The diagram below provides a complete structural overview of the RC4 cipher, illustrating both the key scheduling phase and the pseudorandom generation phase that produces the keystream bytes.



Block Cipher Fundamentals

- Unlike stream ciphers, a **block cipher** encrypts data in **fixed-size chunks**.
- Each block is independently encrypted using the same key, producing a ciphertext block of equal size.

Fixed Block Size

- Data is divided into blocks of a fixed size (e.g., 128 bits for AES).
- Each block is encrypted independently with a fixed-size key.

Padding

If the last block contains fewer bytes than the fixed block size, it is padded with additional bytes to fill the block before encryption.

Common Algorithms

AES (Advanced Encryption Standard) and **DES** (Data Encryption Standard) are the most widely used block cipher algorithms.

Data Encryption Standard (DES)

- **DES** is a **landmark symmetric key block cipher standardized** by the U.S. government in 1977.
- Though now considered **obsolete**, it established the foundation for modern block cipher design.

DES Specifications

- Symmetric key encryption system
- Encrypts **64-bit** data blocks
- Uses a **56-bit** key
- Vulnerable to brute-force due to short key length

Triple DES (3DES)

- As DES became insecure, **3DES** was developed as an interim solution.
- It applies **DES encryption three times** using three different 56-bit keys (K1, K2, K3), forming a "key bundle."
- The U.S. Federal Government later ran a contest to find a full replacement — resulting in AES.

Triple DES (3DES): Options

3DES applies DES three times in an **Encrypt–Decrypt–Encrypt (EDE)** sequence: DES encrypt with K1, DES decrypt with K2, DES encrypt with K3. Three key options exist, each with different security levels.

1

Option 3 — Least Secure

K1 = K2 = K3. All three keys identical — equivalent to single DES.

2

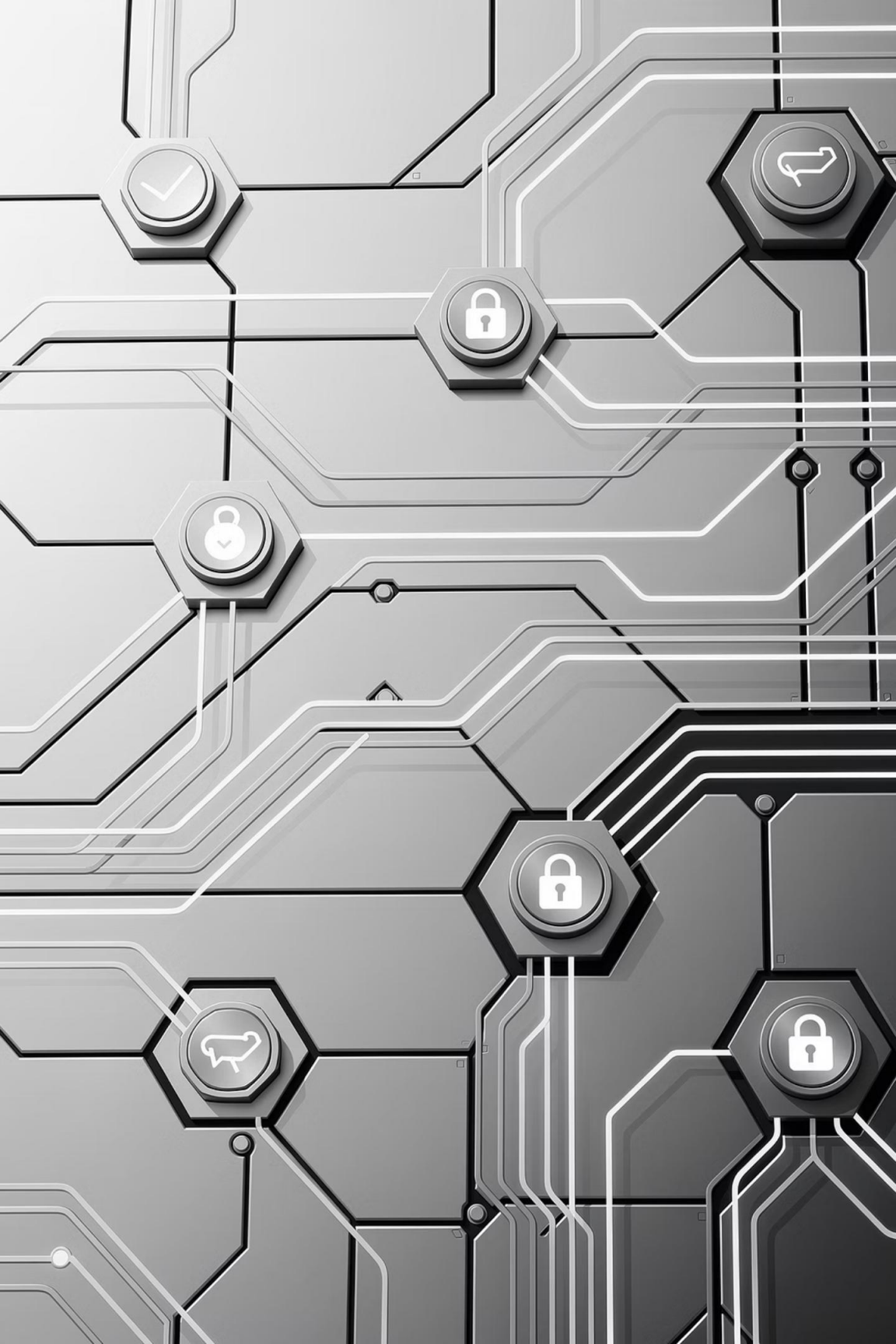
Option 2 — Moderate Security

K1 = K3, but K2 is different. Provides 112-bit effective key strength.

3

Option 1 — Most Secure

K1, K2, and K3 are all independent and different. Provides 168-bit key length — the recommended configuration.



BLOCK CIPHERS

Advanced Encryption Standard (AES)

- AES is the NIST-standardized block cipher that replaced DES, adopted by the U.S. government to protect sensitive unclassified material.
- It is today's dominant symmetric encryption algorithm.

Block Size

Fixed **128-bit** block size for all key variants.

Key Sizes

128-bit (AES-128),
192-bit (AES-192), or
256-bit (AES-256).

Symmetric

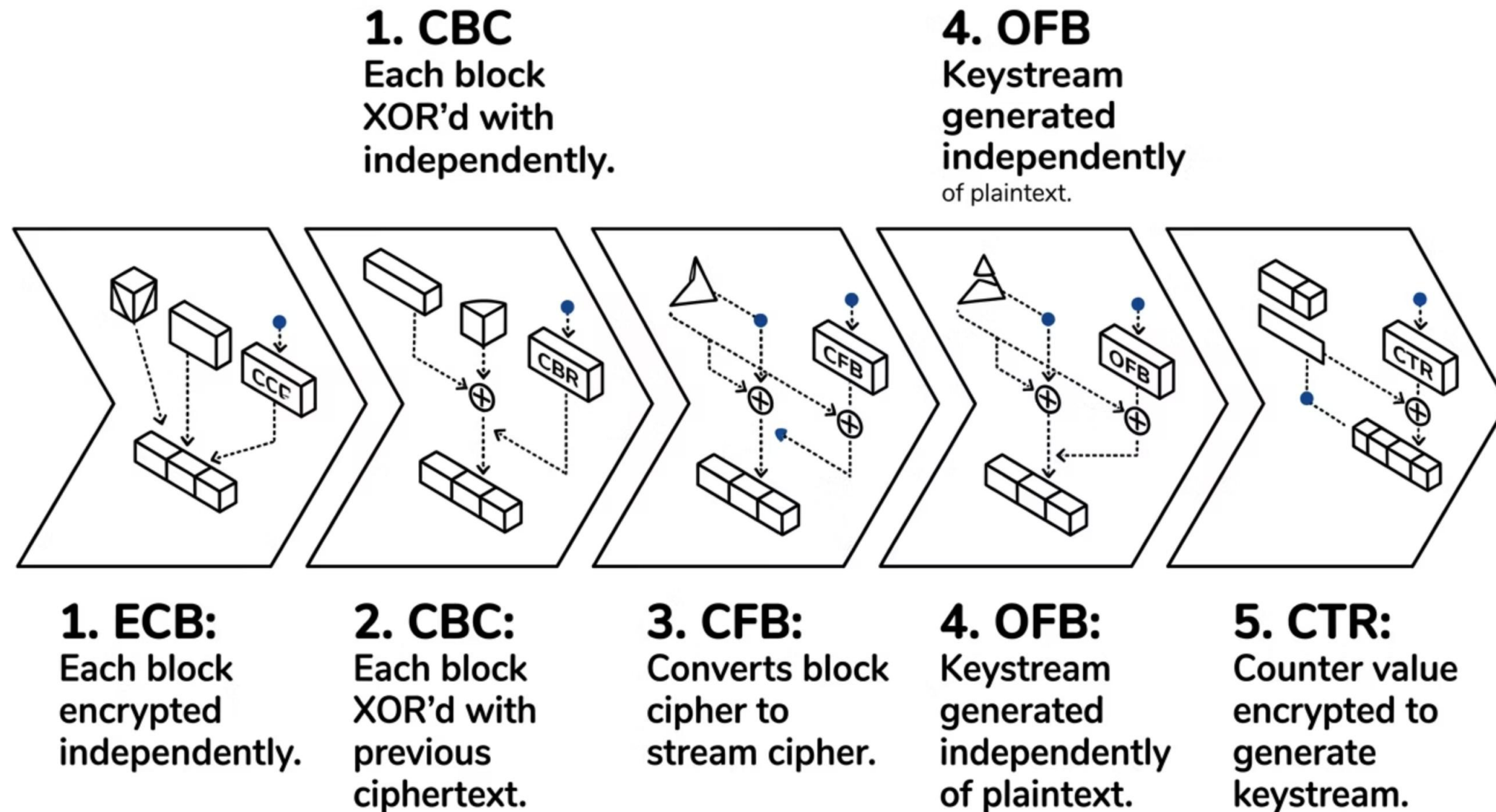
Same key used for both encryption and decryption.

Efficiency

Designed for efficient implementation in both **software** and **hardware**. Used in telecommunications, finance, and government.

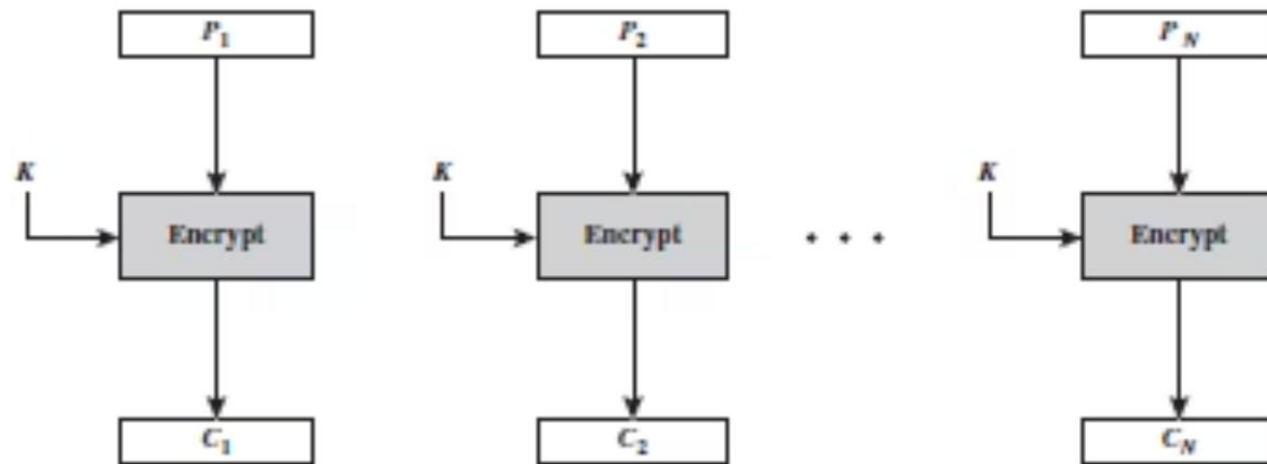
Block Cipher Modes of Operation

When a block cipher is used to encrypt messages longer than a single block, a **mode of operation** defines how successive blocks are processed. Five standard modes have been defined to address different security and performance requirements.

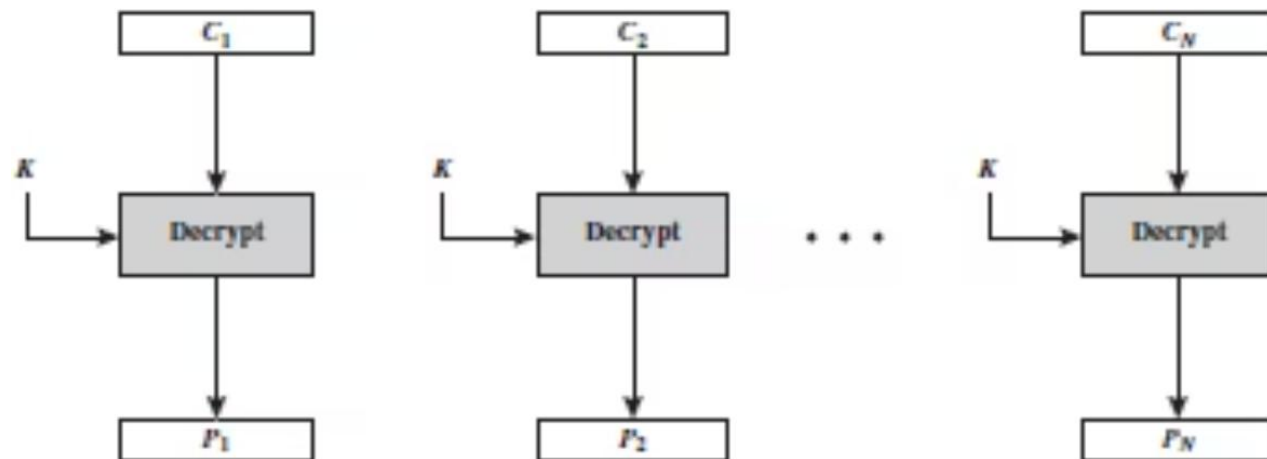


When multiple plaintext blocks are encrypted with the same key, security issues arise — modes of operation are designed to mitigate these vulnerabilities.

Electronic Code Book (ECB) Mode



(a) Encryption



(b) Decryption

How ECB Works

- In ECB mode, each b -bit block of plaintext is encrypted independently using the same key.
- There is no dependency between blocks.

Use Case

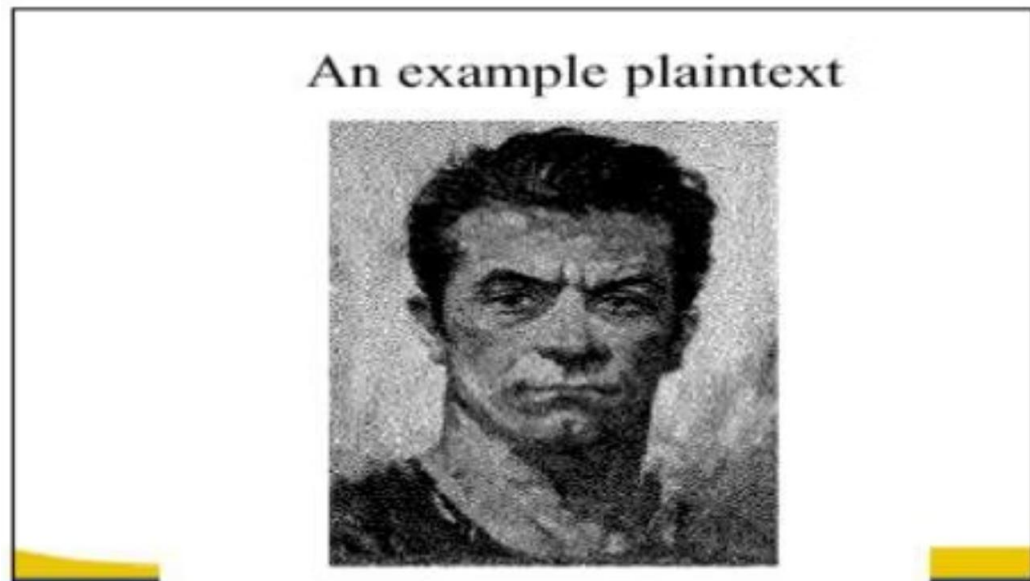
Ideal for encrypting **short, isolated data** such as transmitting AES or DES keys.

Critical Weakness

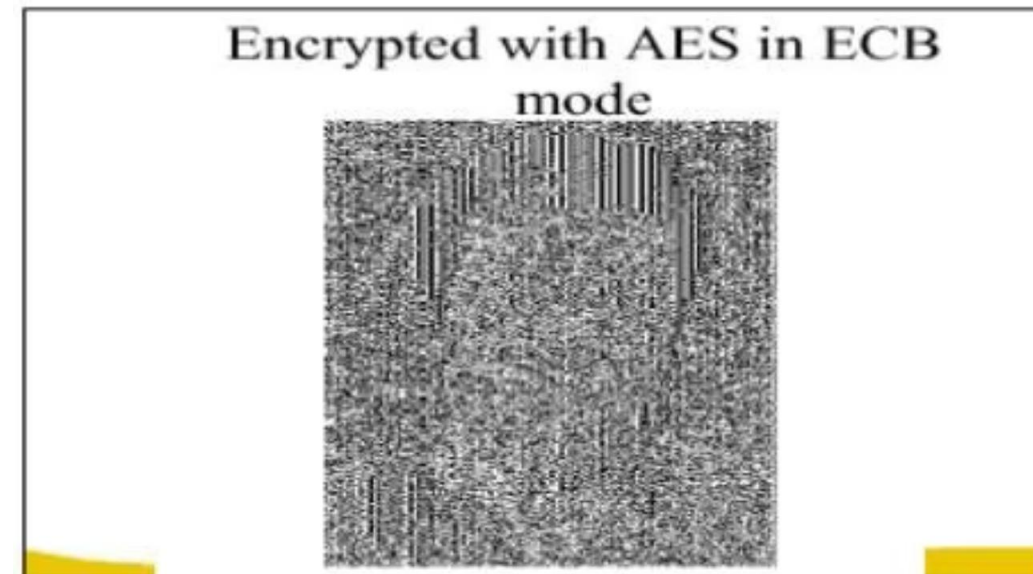
- Identical plaintext blocks **always** produce identical ciphertext blocks.
- For structured or lengthy messages, **this leaks information about patterns in the data — a serious security flaw.**

ECB Weakness: Visual Demonstration

- The following images powerfully illustrate ECB's fundamental flaw.
- When an image is encrypted using ECB mode, block-level patterns in the original data remain visible in the ciphertext — the structural information of the image is preserved despite encryption.



Original image — plaintext



ECB-encrypted — patterns still visible

Cipher Block Chaining (CBC) Mode

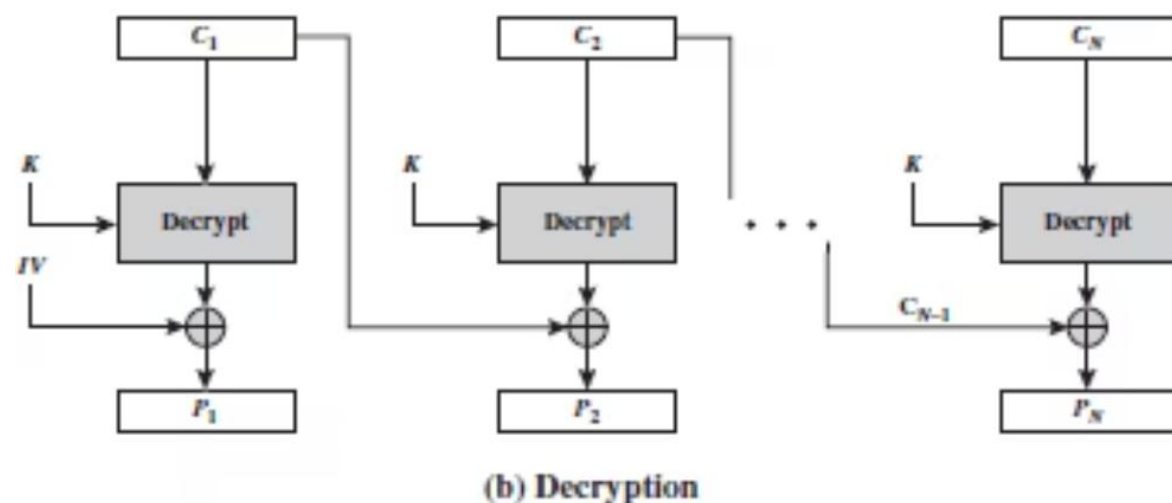
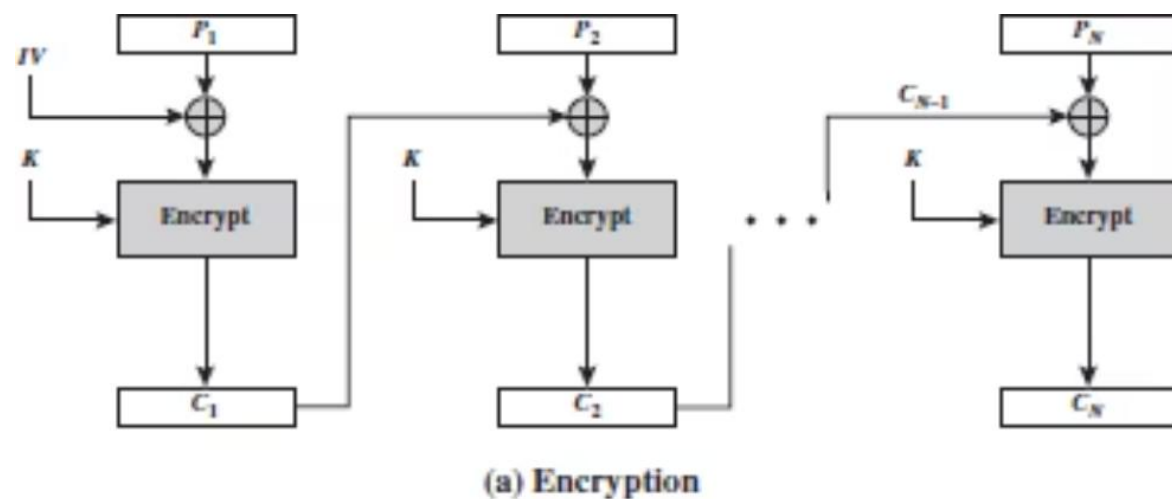


Figure 6.4 Cipher Block Chaining (CFB) Mode

How CBC Overcomes ECB

- CBC XORs each plaintext block with the **previous ciphertext block** before encryption.
- This introduces dependency between blocks, ensuring identical plaintext blocks produce different ciphertext blocks.

Initialization Vector (IV)

$$C_j = E(K, [C_{j-1} \oplus P_j])$$

$$D(K, C_j) = D(K, E(K, [C_{j-1} \oplus P_j]))$$

$$D(K, C_j) = C_{j-1} \oplus P_j$$

$$C_{j-1} \oplus D(K, C_j) = C_{j-1} \oplus C_{j-1} \oplus P_j = P_j$$

The IV must be protected against unauthorized modification — typically transmitted via ECB encryption.

CBC Mode: IV Generation & Applications

IV Generation Methods

- **Nonce:** A unique value for each encryption execution — may be a counter, timestamp, or message number.
- **Random Block:** Generated by a random number generator for unpredictability.

Applications

- Suitable for encrypting messages of length greater than b bits.
- Used for achieving both **confidentiality** and **authentication**.
- Widely used in file encryption and secure communications protocols.

Cipher Feedback (CFB) Mode

Stream-Like Operation

- CFB mode treats the message as a **stream of bits**, eliminating the need to pad the message to a full block boundary.
- This makes it suitable for character-by-character encryption.

Key Properties

- If a character stream is being transmitted, each character can be encrypted and sent immediately — no buffering required.
- The IV plays a critical role in initializing the feedback register.

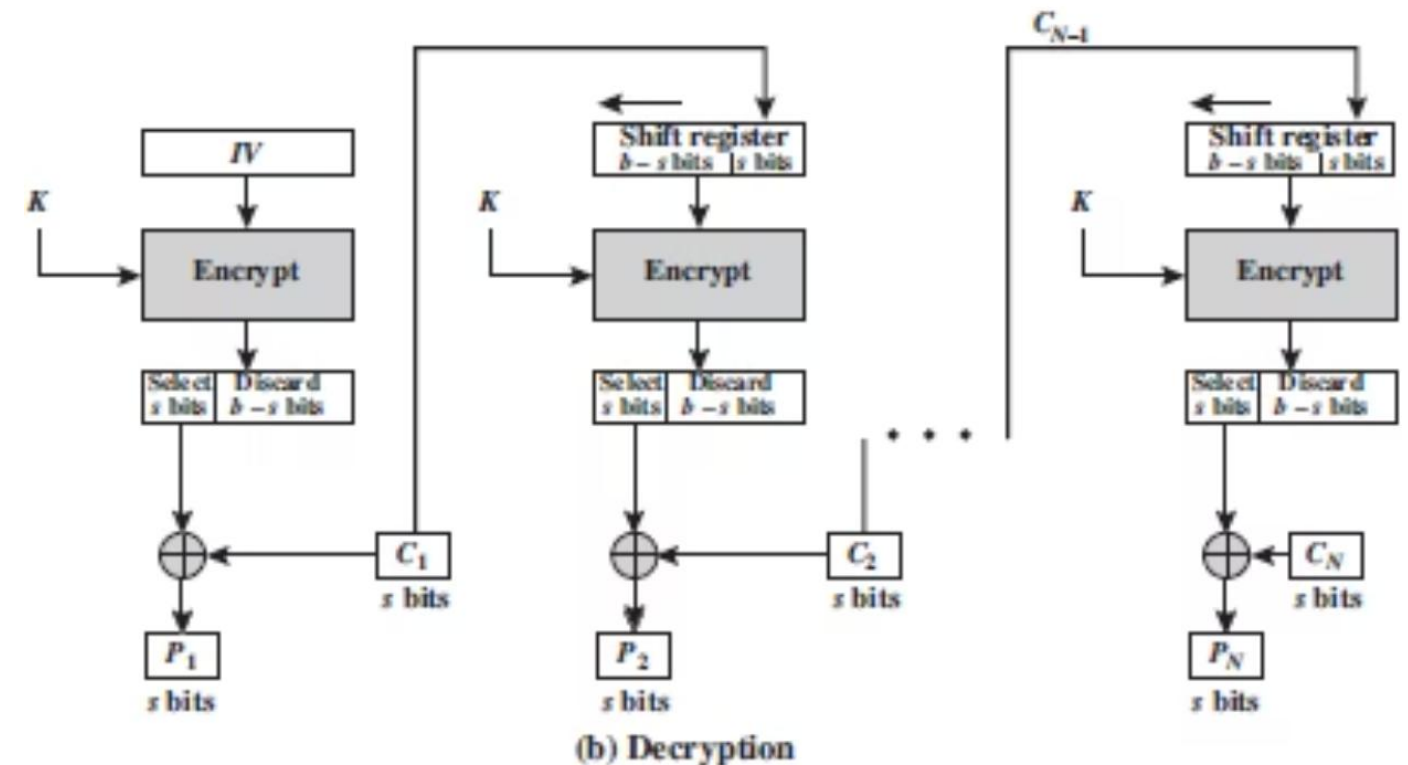
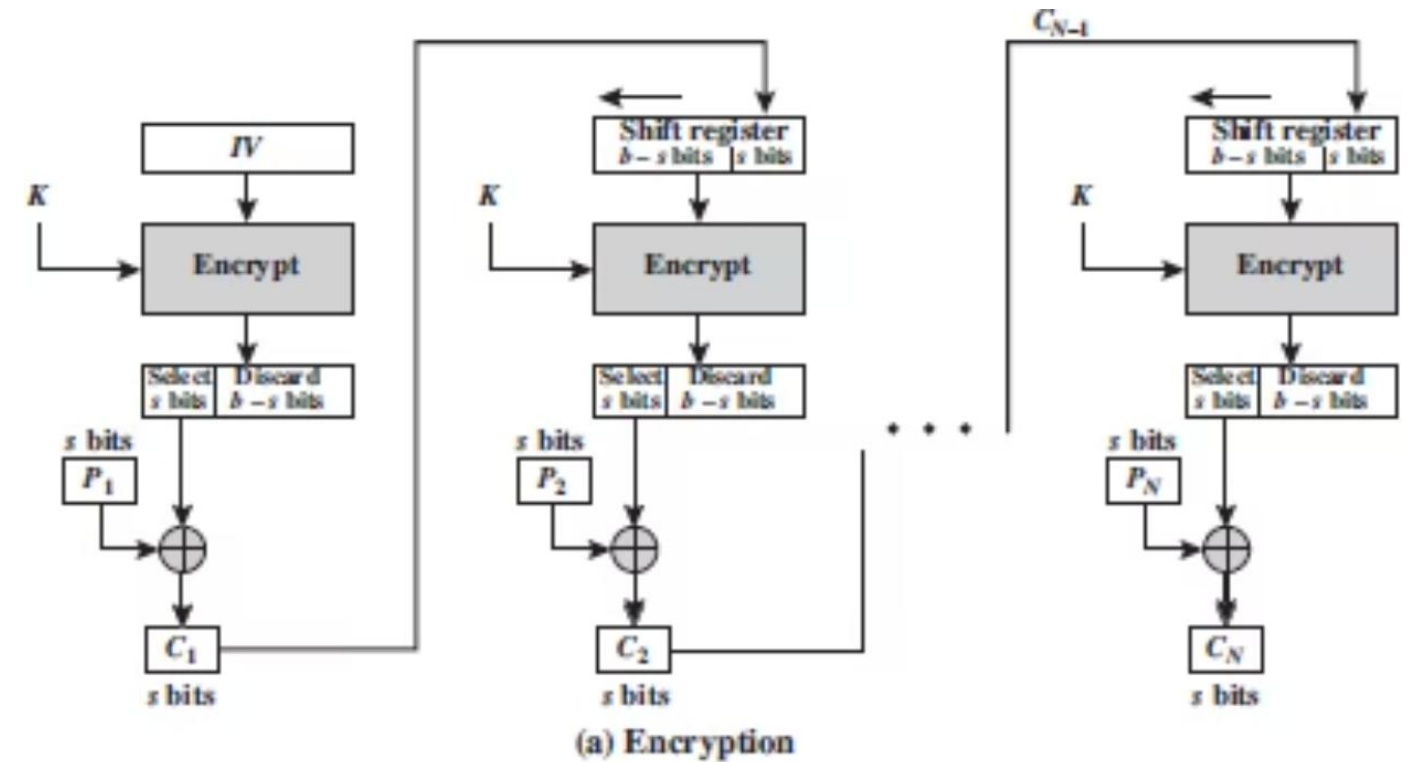


Figure 6.5 s -bit Cipher Feedback (CFB) Mode

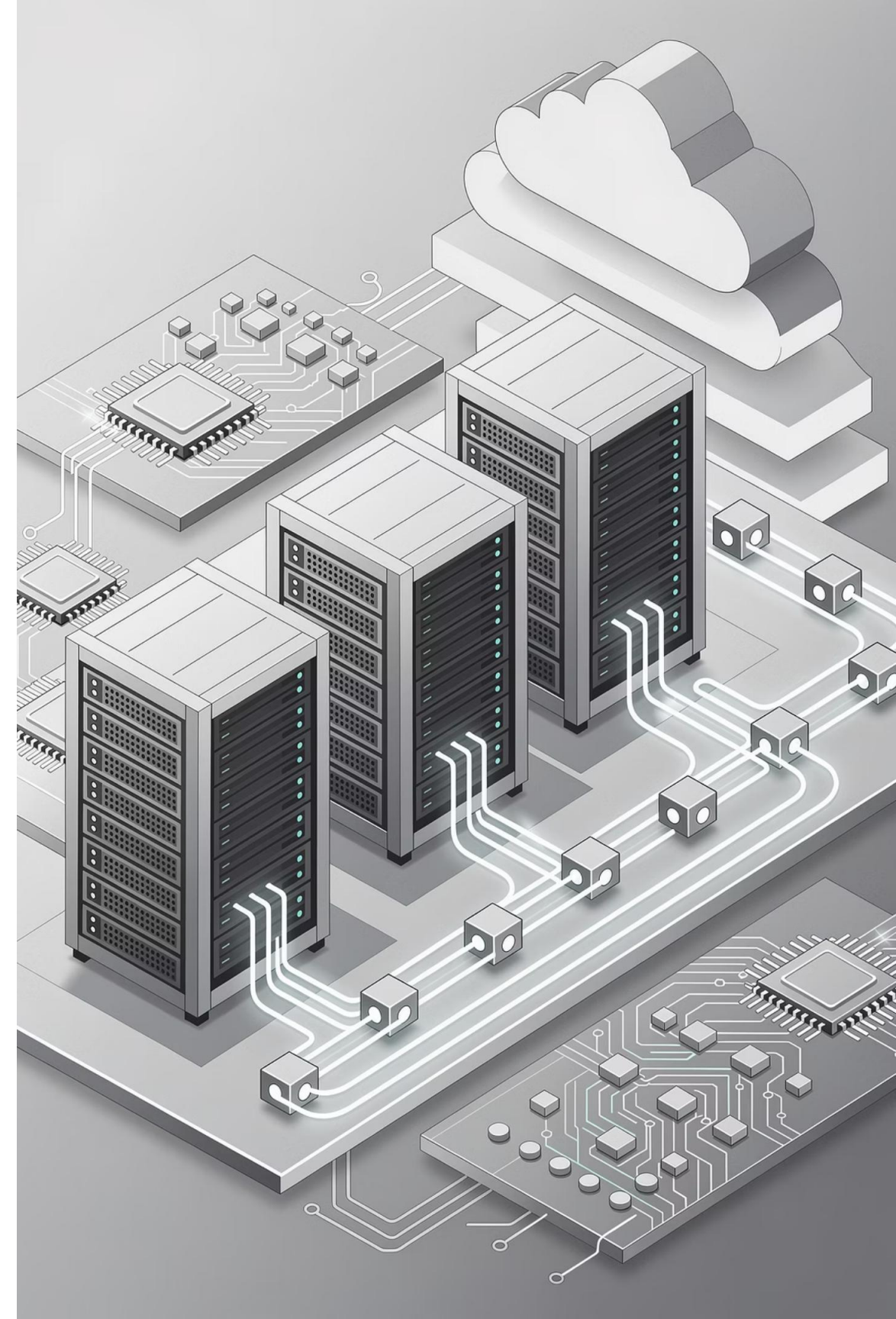
CFB Mode: Parallelism Characteristics

Encryption (Sequential Only)

In CFB encryption, each forward cipher function depends on the result of the **previous** cipher function. Therefore, multiple cipher operations **cannot** be performed in parallel — encryption is inherently sequential.

Decryption (Parallelizable)

In CFB decryption, the forward cipher functions can be performed **in parallel** because the input blocks can be constructed directly from the IV and the received ciphertext, without waiting for previous decryption results.



Output Feedback (OFB) Mode

How OFB Works

- In OFB, the keystream is generated **independently of the plaintext**.
- The output of each cipher operation feeds back into the next cipher input, forming a self-sustaining keystream loop.

Key Properties

- The IV must be a **unique nonce** per encryption operation — **reuse exposes plaintext**.
- Bit errors in transmission **do not propagate** — an error in C_1 only affects recovery of P_1 .
- Partial last blocks use the most significant u bits of the final output block.

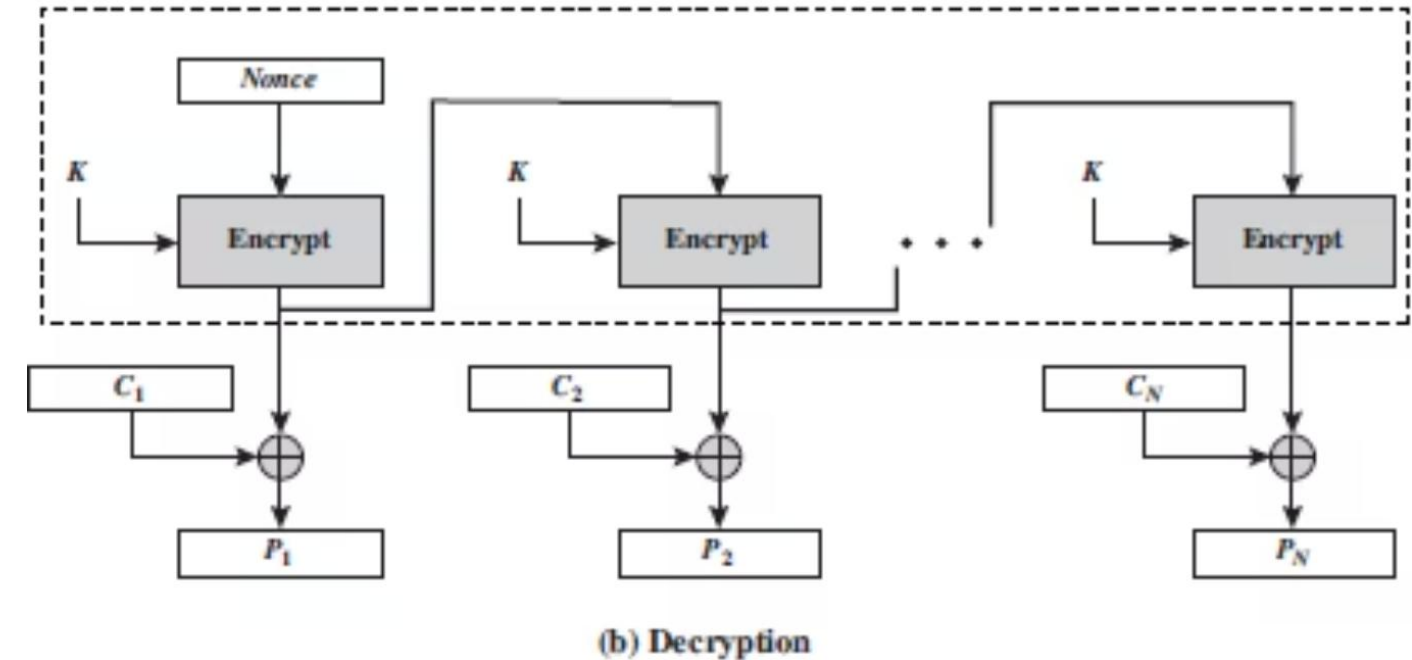
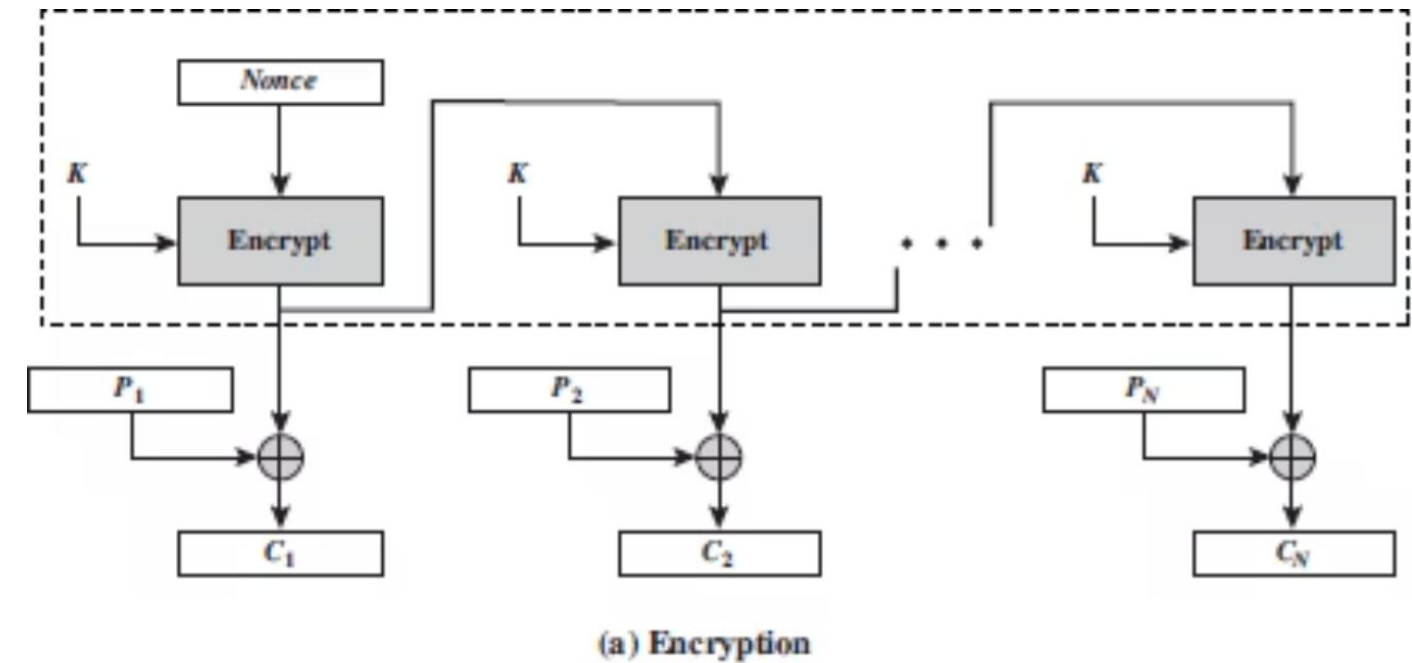


Figure 6.6 Output Feedback (OFB) Mode

Counter (CTR) Mode

How CTR Works

- CTR mode encrypts successive counter values ($T_1, T_2, \dots, T_N$) to produce keystream blocks, which are then XOR'd with plaintext blocks.
- Each counter value must be unique (nonce) across all messages.

Security Note

- If a counter value is known and a plaintext block encrypted with that counter is identified, the corresponding encryption output is recoverable — allowing other plaintext blocks encrypted with the same counter to be exposed.

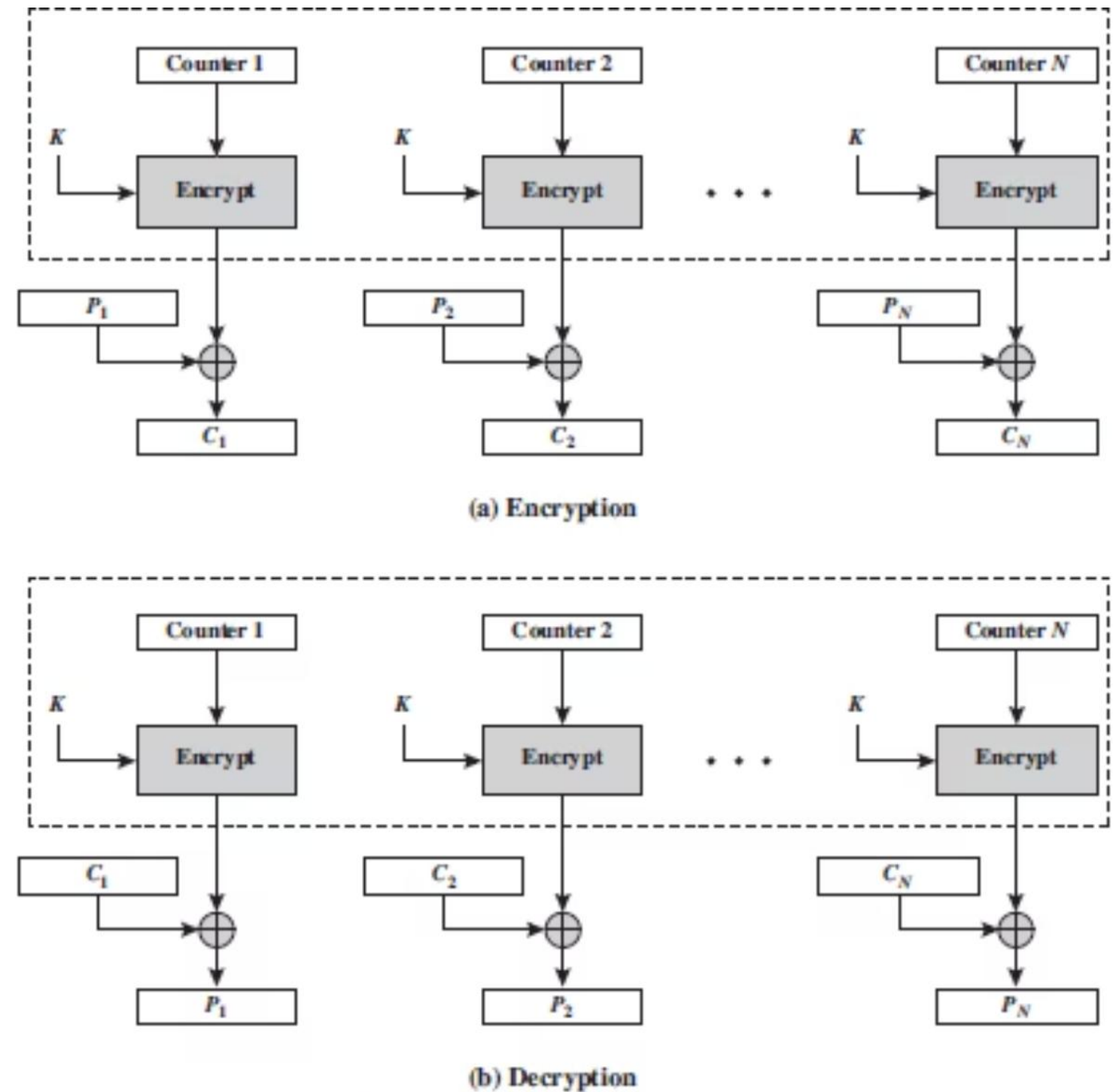


Figure 6.7 Counter (CTR) Mode

CTR	$C_j = P_j \oplus E(K, T_j) \quad j = 1, \dots, N - 1$ $C_N^* = P_N^* \oplus \text{MSB}_m[E(K, T_N)]$	$P_j = C_j \oplus E(K, T_j) \quad j = 1, \dots, N - 1$ $P_N^* = C_N^* \oplus \text{MSB}_m[E(K, T_N)]$
-----	---	---

CTR Mode: Advantages

CTR mode offers significant practical advantages over other block cipher modes, making it the preferred choice in modern high-performance environments such as TLS 1.3 and disk encryption.



Full Parallelism

Both encryption and decryption can be performed **in parallel** — all blocks can be processed simultaneously, enabling high throughput.



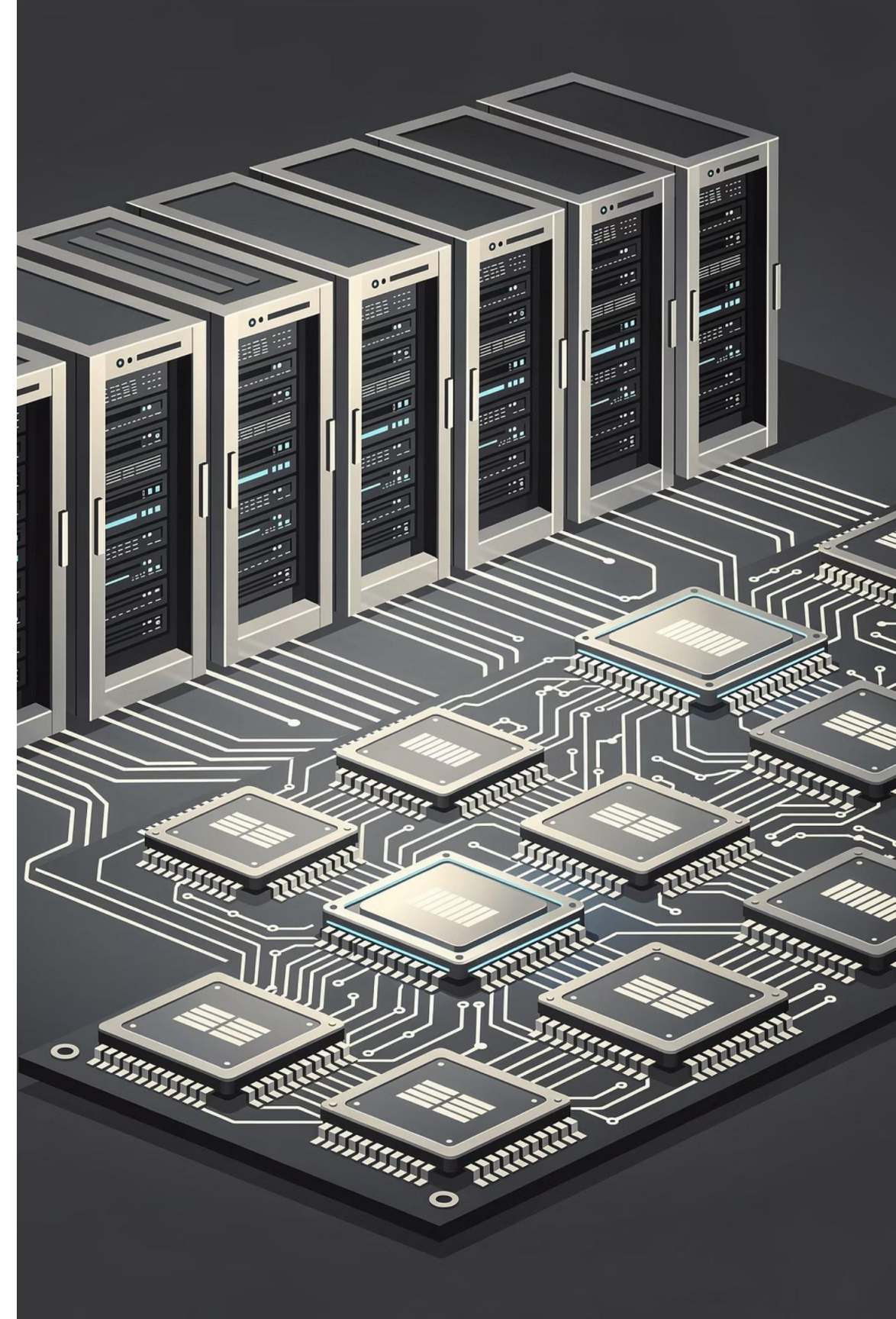
Preprocessing

The keystream can be **precomputed** before the plaintext is available, reducing encryption latency in time-critical applications.



Encryption Only

Unlike ECB and CBC, CTR mode requires **only the encryption algorithm** — the decryption cipher function is never needed, simplifying implementation.



References & Further Reading

Primary Textbook

Stallings, W. (2023). *Cryptography and Network Security: Principles and Practice* (8th ed., Global ed.). Pearson.

The canonical reference for all cryptography elements covered in this lecture.

DNSSEC Resource

Cloudflare. (n.d.). *How DNSSEC works*.

<https://www.cloudflare.com/dns/dnssec/how-dnssec-works/>

Lecture slides 64–73 are adapted from this source. Recommended for supplementary reading on applying cryptography to DNS security.